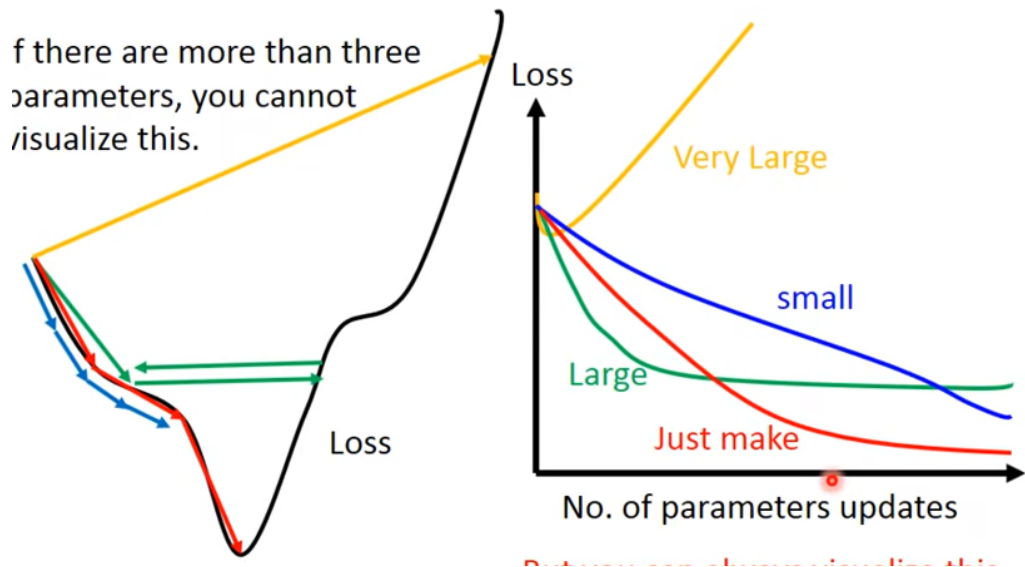


- ML (Machine Learning)机器学习
- Regression 回归 预测连续数值问题
<https://www.bilibili.com/video/BV1JE411g7XF?t=3861.5&p=3>
 - Logistic Regression 逻辑回归
- Basic Concept基本概念
 - feature特征
 - variable变量
 - parameters参数
 - error误差=真实值-预测值
 - mean平均值
 - bias偏差（偏置）
 - variance方差
 - Mean Squared Error均方误差
 - Expectation期望
 - Probability Distribution概率分布
 - Maximum Likelihood Estimation最大似然估计
- <https://www.bilibili.com/video/BV1JE411g7XF?t=849.2&p=4>
 - Gradient Descent梯度下降
 - Stochastic Gradient Descent（SGD）= 随机梯度下降
 - update更新（调整参数）
 - learning rate学习率



- Optimizer优化器

优化器	英文全称	中文
SGD	Stochastic Gradient Descent	随机梯度下降
Momentum	Momentum Optimizer	动量优化器
AdaGrad	Adaptive Gradient Algorithm	自适应梯度优化器
RMSProp	Root Mean Square Propagation	均方根传播优化器
Adam	Adaptive Moment Estimation	自适应矩估计优化器

- Feature Scaling 特征缩放
 - normalization/Min-Max Scaling归一化
 - Standardization/Z-score Scaling标准化
- Classification 分类问题
 - binary classificationstion二分类
 - multi-class classification多分类
- Optimization for Deep Learning 深度学习优化算法

阶段	英文课程	中文名称	主要内容	难度	重要程度
第三阶段：进入深度学习	Brief Introduction of Deep Learning	深度学习简介	神经网络基本思想	☆☆	★★★★★
	Backpropagation	反向传播算法	神经网络训练核心算法	★★★★	★★★★★
	Tips for Training DNN	深度神经网络训练技巧	初始化、BN、Dropout等	★★★★	★★★★★
	Why Deep	为什么要使用深度学习	深层网络优势分析	☆☆☆	★★★★★
第四阶段：PyTorch 开发	PyTorch Tutorial	PyTorch 教程	Tensor、Autograd、模型搭建	☆☆	★★★★★
	CNN Convolutional Neural Network	卷积神经网络（CNN）	图像识别、目标检测	★★★★	★★★★★
	Recurrent Neural Network (Part I)	循环神经网络（一）	序列数据处理	★★★★	★★★★★
	Recurrent Neural Network (Part II)	循环神经网络（二）	LSTM、GRU等	★★★★	★★★★★
	Graph Neural Network (1,2)（选学）	图神经网络	图结构数据学习	★★★★★	★★★★★
	Unsupervised Learning – Word Embedding	无监督学习——词嵌入	Word2Vec、Embedding	★★★★	★★★★★

- regression输出为数值
-
- generation生成
- Labeled Data 有标签数据 / 已标注数据
- Data Labeling 数据标注
- Function函数
- loss function损失函数
- supervised learning监督学习
- unsupervised learning无监督学习

- reinforcement learning 强化学习（reward 奖励）
- transfer learning 迁移学习
- meta learning 元学习
- life-long learning/Continuous Learning/Never ending learning/incremental learning 终身学习

CNN卷积神经网络

Input Image 输入图像
Convolution 卷积
Activation Function 激活函数
Pooling Layer 池化层
Flatten 展平
Fully Connected 全连接层
Output 输出层

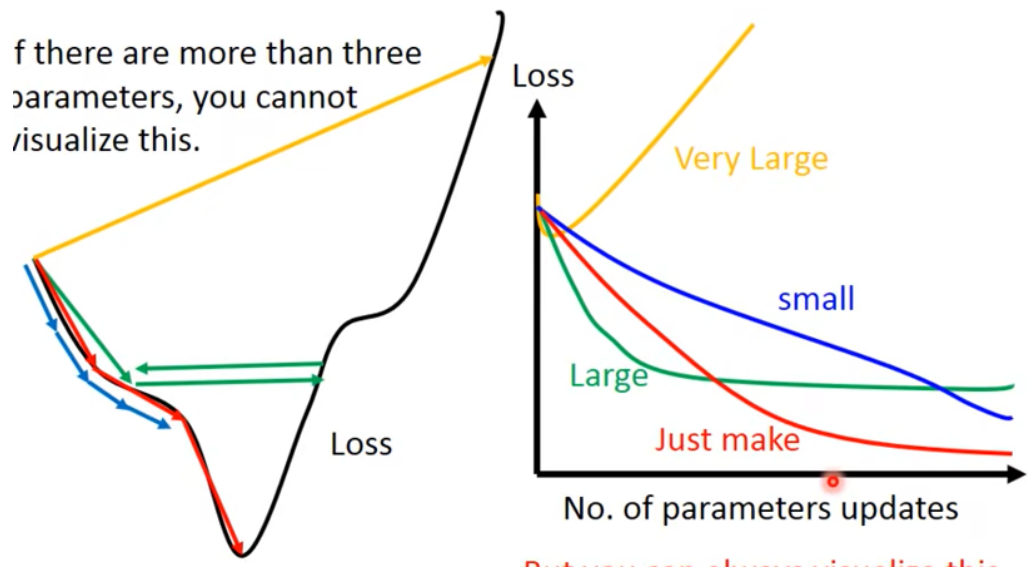
- subsampling 减少数据数量，同时尽量保留重要信息
- Convolution 卷积
- kernel 卷积核
- Pooling 池化
- Flatten 展平，把多维数据转换成一维向量，方便送入全连接层
- Feature Map 特征图
- layer 层
- property 特征，属性
- uprate 更新
- matrix 矩阵
- filter 卷积核
- element 元素
- parameters 参数
- pattern 图像

英文		中文翻译
ML	ML	机器学习（Machine Learning）
MLP	Regression - Case Study	回归案例研究
MLP	Basic Concept	基础概念
MLP	Gradient Descent 1/2/3	梯度下降（一/二/三）
MLP	Optimization for Deep Learning	深度学习优化方法
MLP	Classification	分类
MLP	Logistic Regression	逻辑回归
MLP	Brief Introduction of Deep Learning	深度学习简介
MLP	Backpropagation	反向传播

	英文	中文翻译
MLP	Tips for Training DNN	训练深度神经网络的技巧
MLP	Why Deep Learning	为什么选择深度学习
MLP	PyTorch Tutorial	PyTorch教程
CNN	(CNN)Convolutional Neural Network	卷积神经网络
GNN	(GNN)Graph Neural Network	图神经网络
RNN	(RNN)Recurrent Neural Network	循环神经网络

MLP

- ML (Machine Learning)机器学习
- Regression 回归 预测连续数值问题
<https://www.bilibili.com/video/BV1JE411g7XF?t=3861.5&p=3>
 - Logistic Regression 逻辑回归
- Basic Concept基本概念
 - feature特征
 - variable变量
 - parameters参数
 - error误差=真实值-预测值
 - mean平均值
 - bias偏差（偏置）
 - variance方差
 - Mean Squared Error均方误差
 - Expectation期望
 - Probability Distribution概率分布
 - Maximum Likelihood Estimation最大似然估计
<https://www.bilibili.com/video/BV1JE411g7XF?t=849.2&p=4>
- Gradient Descent梯度下降
- Stochastic Gradient Descent（SGD）= 随机梯度下降
 - update更新（调整参数）
 - learning rate学习率



- Optimizer 优化器

优化器	英文全称	中文
SGD	Stochastic Gradient Descent	随机梯度下降
Momentum	Momentum Optimizer	动量优化器
AdaGrad	Adaptive Gradient Algorithm	自适应梯度优化器
RMSProp	Root Mean Square Propagation	均方根传播优化器
Adam	Adaptive Moment Estimation	自适应矩估计优化器

- Feature Scaling 特征缩放
 - normalization/Min-Max Scaling 归一化
 - Standardization/Z-score Scaling 标准化
- Classification 分类问题
 - binary classification 二分类
 - multi-class classification 多分类
- Optimization for Deep Learning 深度学习优化算法
- regression 输出为数值
- generation 生成
- Labeled Data 有标签数据 / 已标注数据
- Data Labeling 数据标注
- Function 函数
- loss function 损失函数
- supervised learning 监督学习
- unsupervised learning 无监督学习
- reinforcement learning 强化学习 (reward 奖励)
- transfer learning 迁移学习
- meta learning 元学习
- life-long learning/Continuous Learning/Never ending learning/incremental learning 终身学习

CNN(卷积神经网络)

Input Image输入图像

Convolution卷积

Activation Function激活函数

Pooling Layer池化层

Flatten展平

Fully Connected全连接层

Output输出层

- subsampling减少数据数量，同时尽量保留重要信息
- Convolution卷积
- kernel卷积核
- Pooling池化
- Flatten 展平，把多维数据转换成一维向量，方便送入全连接层
- Feature Map特征图
- layer层
- property特征，属性
- uprate更新
- matrix矩阵
- filter卷积核
- element元素
- parameters参数
- pattern图像

RNN(循环神经网络)

- 1-of-N encoding
- slot filling槽位填充
- store存储
- memory
- initial values起始值
- bias偏置
- activation functuon激活函数
- linear线性
- sequence序列
- LSTM (Long SHORT-term Memory) 长短期记忆网络
- Memory cell记忆单元
- input gate
- output gate

- forget gate
- each word is represented as a vector : 表示成一个向量
- probability distribution that the input word belonging to the slots输入单词属于各个 slot (槽位) 的概率分布
- dimension维度

MINS数据集特点:

训练集60,000张, 测试集10,000张

[将训练集中分出10,000张作为验证集]

数据集处理:

将图片数据从二进制→三维数组 (通道数, 宽, 高)

批次为大小 (64)

将像素归一化 ($x1=x/255$)

将标签数据从二进制→整数标签→one-hot编码

将三维数组转为二维数组 (通道, 宽×高)

初始化 (He初始化) 权重和偏置设置

$x2=权重w1转置 \times 二维数组像素值 + 偏置$

神经元为每个输入分配一个权重, 有多少个输入就有多少个权重, 一个神经元会接收上一层的所有输入值, 并且为每一个输入分配一个独立的权重。不同神经元的区别就在于它们拥有不同的权重组合和偏置。

接着设计第一层神经元个数为256个

对x2加relu激活函数= $z1$

$z2=权重w2转置 \times z1 + 偏置$

接着设计第二层神经元个数为256个

对z2加relu激活函数= $z3$

输出层: 将隐藏层提取出的特征映射到类别

softmax将输出层的得分转换成概率

损失函数 (交叉熵损失函数) 告诉错了多少

梯度告诉怎么改

学习率(0.001)告诉调整的幅度

优化器 (adam) 实现权重, 偏置的更新

训练轮数: 45

早停机制: 耐心值10

随机种子: 42

评价指标: acc, f1

CNN 通过有监督训练学习图像的层次化视觉表征, 以实现多分类任务。训练的评估标准是损失函数 (通常为交叉熵), 它衡量模型输出的概率分布与真实标签分布之间的差异。

数据获取与处理

- 从数据库下载/使用指定路径, 获取数据。数据包含图片和标签, 都是二进制, 分训练集和测试集
- 将二进制图片使用numpy的功能处理, 得到三维像素数组(C, H, W) (通道, 高, 宽)
- 再经维度变换与归一化, 将三维像素数组转为 CNN 可处理的四维批次张量 (B, C, H, W) (每个批次样本数, 通道数, 高, 宽), 设为 x_i [因为CNN是一批一批图像处理, 每批中的图片同时进行]
- 标签则从二进制解析为整数索引或One-Hot 向量。

卷积层

使用线性变换
卷积核相当于 w
卷积的过程相当于 $w*x$ 的过程
再对卷积后的结果加上一个偏置 b
得到 $wx+b$

BatchNorm

因为一次处理一批，所以**BatchNorm**批数据进行求平均等多种方法得到最终具有代表性的数

激活层（用的ReLU激活函数）

接下来进入激活层
使用ReLU激活函数，得到 $ReLU(wx+b)$,即求得非线性特征

池化层

接着进入池化层，降低特征分辨率（让特征图的尺寸（高和宽）变小，使每个像素点"代表"更大范围的原始图像区域。）

展平层

再进入展平层将数组转为一维

全连接层

再到达全连接层映射回得分，每个输入神经元与每个输出神经元之间都有独立的权重， W 是权重矩阵，通过整向量矩阵乘法计算，也有偏置

Softmax激活函数

最终将得分用softmax激活函数求出预测概率，`return`后面是softmax的表达式

损失函数

接着通过损失函数判断预测能力
如果预测效果不好

反向传播

梯度，复合函数求导
则反向传播逐层返回，重新调整 w 和 b
而决定 w 和 b 如何调整是从损失函数出发，根据梯度下降趋势来判断，加以优化器来优化实现参数更新

整个过程由 **Dropout**、数据增强、权重衰减、**BatchNorm**等防止过拟合

0 学习目的

- 进行深度学习方面的实践，构建神经网络，超参数调整，正则化，诊断偏差，方差，高级优化算法
- 结构化机器学习工程
- 卷积神经网络，序列模型（RNN-LSTM），如何应用于自然语言处理（NLP）

1 二分类任务

二分类任务的目标是训练出来一个分类器

1.1 参数

图片特征向量的维度:

$$n_x = size \times \text{通道数}$$

输入:

$$x \in R^{n_x}$$

输出:

$$y \in (0, 1)$$

样本的表示:

$$(x^i, y^i)$$

训练样本集合的表示:

$$D = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$$

训练样本的个数(m):

$$m = m_{train}$$

X的矩阵大小

$$n_x \times m$$

$$\mathbf{X} = \begin{bmatrix} 1 & 1 & \dots & 1 \\ \mathbf{x}^{(1)} & \mathbf{x}^{(2)} & \dots & \mathbf{x}^{(m)} \\ 1 & 1 & \dots & 1 \end{bmatrix}$$

$\xleftarrow{\quad \mathbf{m} \quad} \xrightarrow{\quad}$

$\begin{matrix} \uparrow \\ \mathbf{n}_x \\ \downarrow \end{matrix}$

注： $X \in R^{n_x \times m}$

Python命令：

$$X.shape = (n_x, m)$$

Y的矩阵大小为：

$$1 \times m$$

$$\mathbf{Y} = [\mathbf{y}^{(1)} \quad \mathbf{y}^{(2)} \quad \cdots, \mathbf{y}^{(m)}]$$

Python命令:

$$Y.shape = (1, m)$$

1.2 logistic回归算法

logistic是一个用于二分分类的算法

预测值返回的一般不是布尔值，而是对正样本进行预测的可能性概率：

$$\hat{y} = P(y = 1|x), \hat{y} \in [0, 1]$$

Logistic回归的参数（parameters）是：

$$w, w \in R^{n_x} b, b \in R$$

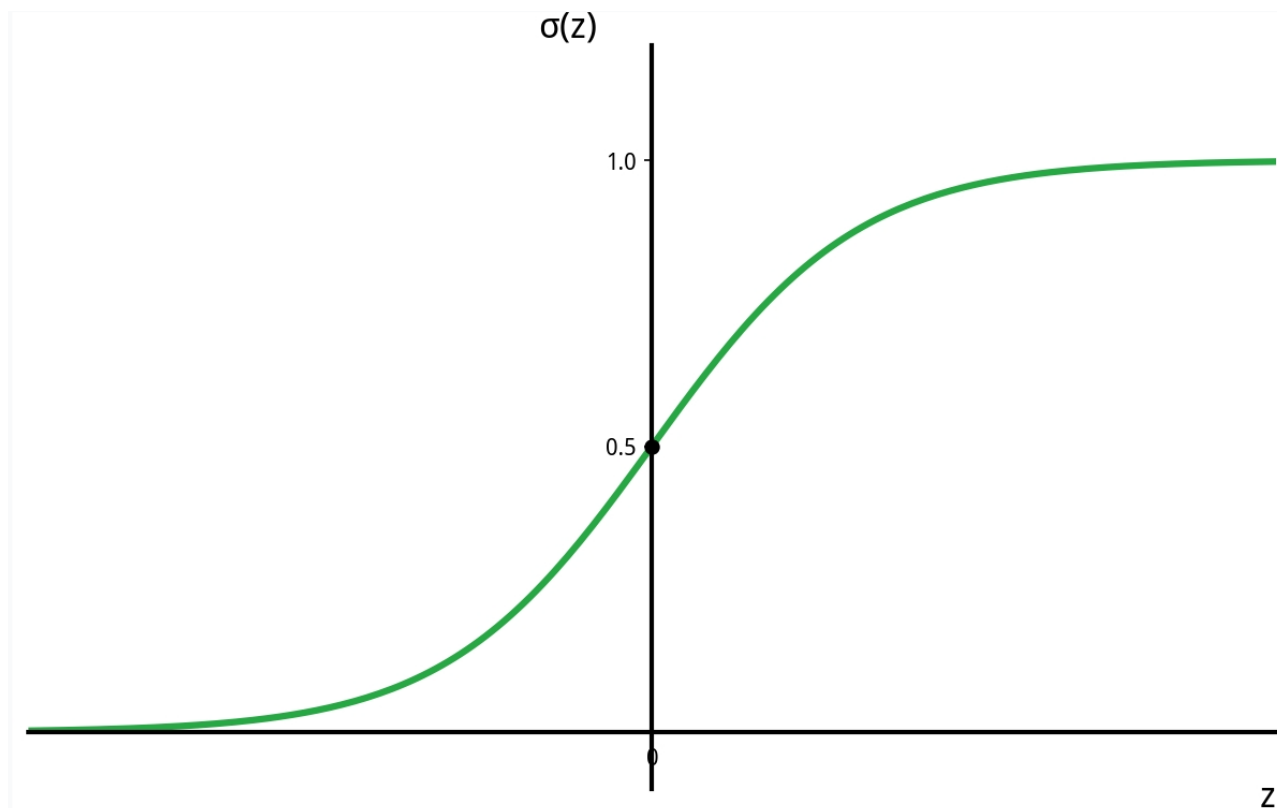
Output:

$$\hat{y} = w^T x + b$$

1.2.1 激活函数（sigmoid函数）

为了实现预测值介于[0,1]范围内，我们使用激活函数（sigmoid函数）作用于该函数，即

$$\hat{y} = \sigma(w^T x + b)$$



sigmoid函数：

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

即：

$$\hat{y} = \frac{1}{1 + e^{-(w^T x + b)}}$$

1.2.2 损失函数&成本函数

训练达到好的效果意味着损失函数和成本函数的值尽可能小

- 损失函数衡量的是单个训练样本上的表现
- 成本函数衡量的是全体训练样本上的表现

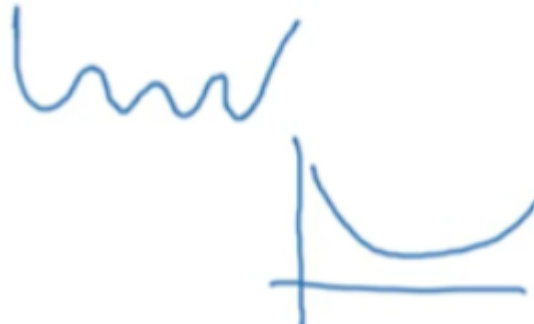
训练集的目的，是为了达到通过训练，调整参数w和b,使得预测结果接近实际结果（0/1）

$$\hat{y}^{(i)} \approx y^{(i)}$$

损失函数（误差函数）[Loss(error) function]，起着与误差平方相似作用

$$L(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2$$

因为使用误差平方会使得梯度下降可能只能找到局部最优但找不到全局最优解



所以选择一种损失函数：

$$L(\hat{y}, y) = - (y \log \hat{y} + (1 - y) \log (1 - \hat{y}))$$

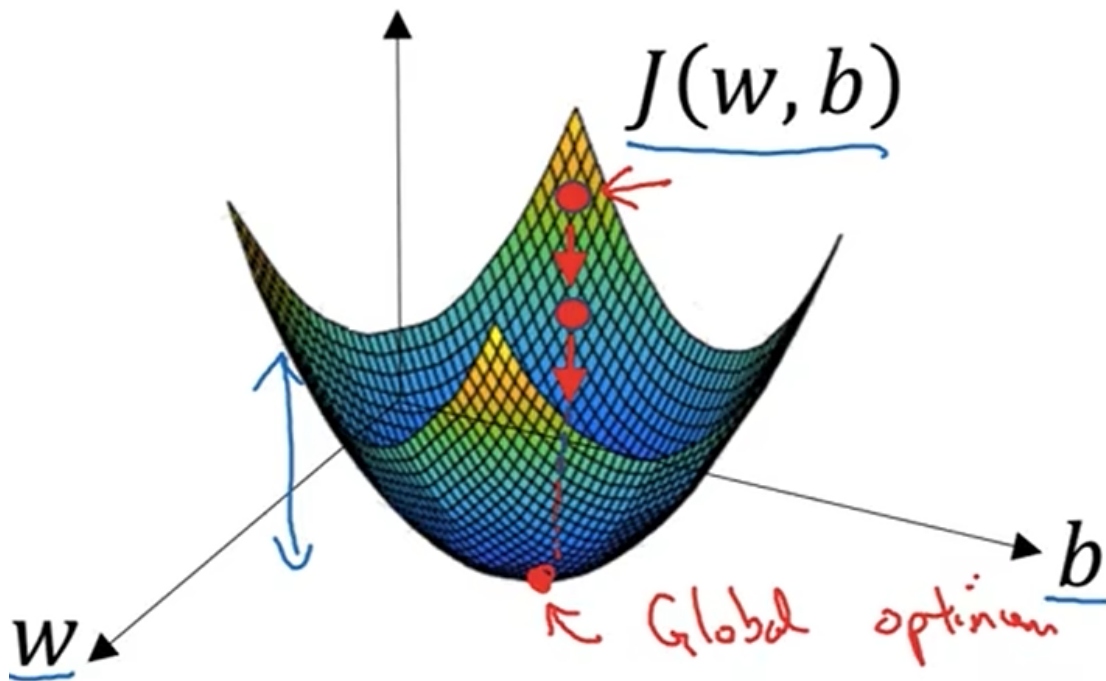
计算机中 $\log$ 默认是以 e 为底数的对数，即 $\ln$

If $y=1$: $L(\hat{y}, y) = -\log \hat{y} \leftarrow$ Want $\log \hat{y}$ large, want $\hat{y}$ large.

If $y=0$: $L(\hat{y}, y) = -\log (1 - \hat{y}) \leftarrow$ Want $\log (1 - \hat{y})$ large ... want $\hat{y}$ small

成本函数（Cost function）：

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log (1 - \hat{y}^{(i)}) \right]$$



1.2.3 优化器（梯度下降）

为达到成本函数 $J(w, b)$ 尽可能小的目的，由成本函数式中相关性得，要训练 w, b 这两个变量（参数）
 w, b 的更新（“:=”理解为赋值）

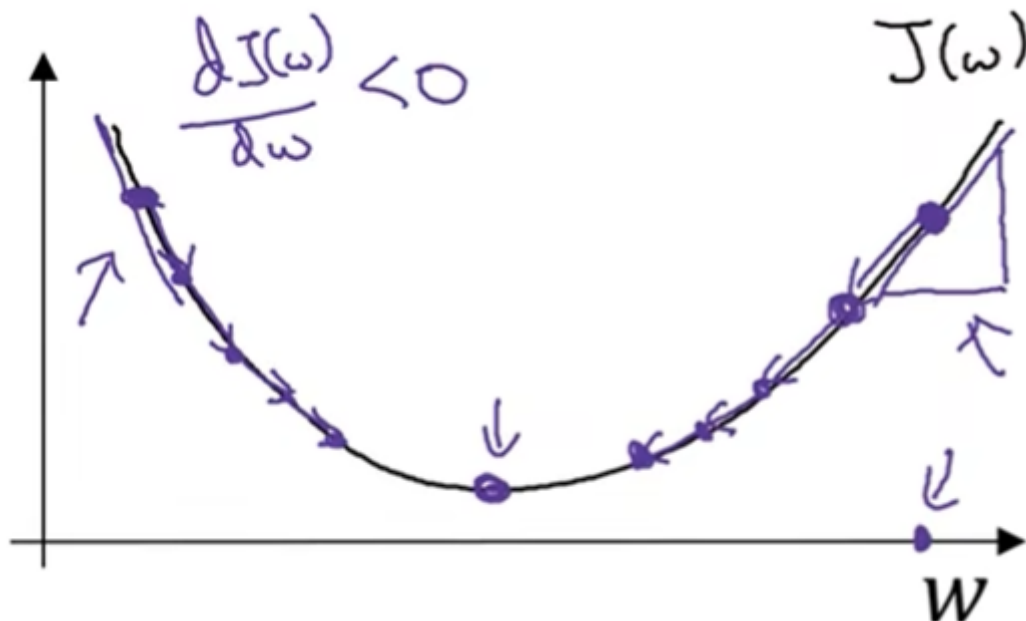
$$w : w - \alpha \frac{\partial J(w, b)}{\partial w}$$

$$b : b - \alpha \frac{\partial J(w, b)}{\partial b}$$

具体训练过程以忽略 b ，为例：

$$w := w - \alpha \frac{dJ(w)}{dw}$$

- $\frac{dJ(w)}{dw}$ 实则是对以变量为 w 的函数 $J(w)$ 进行求导，导为线上点的斜率
- α : *learning rate* (lr) , 学习率, 可以控制每一次或者迭代梯度下降法中的步长



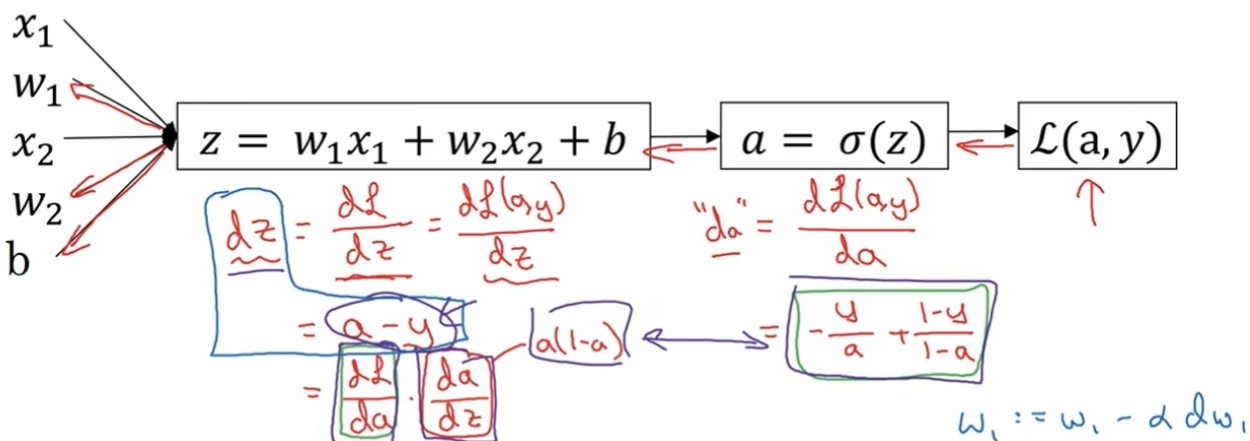
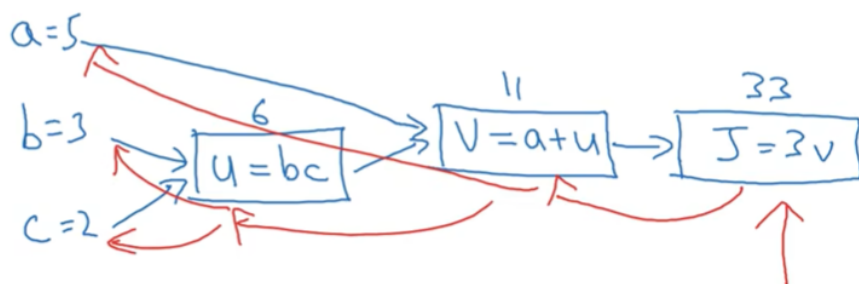
点的移动方向是向最终 $j(w)$ 最小值移动的。斜率是标量，带正负号。

1.2.4 反向传播

反向传播能够解释偏导数是如何计算的，进而能够实现logistic回归的梯度下降法
复合函数求偏导（红色）

$$J(a, b, c) = 3(a + \underbrace{bc}_u) = 3(\underbrace{5 + 3 \times 2}_v) = 33$$

$$\begin{aligned} u &= bc \\ v &= a + u \\ J &= 3v \end{aligned}$$



$$\begin{aligned}
 z^{(i)} &= w^T x^{(i)} + b \\
 \hat{y}^{(i)} &= \sigma(z^{(i)}) \\
 \mathcal{L}(\hat{y}, y) &= -(y \log \hat{y} + (1-y) \log(1-\hat{y})) \\
 &= -(y \ln \hat{y} + (1-y) \ln(1-\hat{y})) \\
 J(w, b) &= \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) \\
 \text{即: } J(w, b) &= \frac{1}{m} \sum_{i=1}^m \left\{ -[y \log \sigma(w^T x^{(i)} + b) + (1-y) \log [1 - \sigma(w^T x^{(i)} + b)]] \right\}
 \end{aligned}$$

$$\begin{aligned}
 dz^{(i)} &= x^{(i)} dw \quad dz^{(i)} = db \\
 d\hat{y}^{(i)} &= d\sigma(z^{(i)}) dz^{(i)} \\
 dL &= -\left(\frac{y}{\hat{y}} \cdot d\hat{y} + \frac{1-y}{1-\hat{y}} (-d\hat{y})\right) \\
 dJ &= \frac{1}{m} \sum_{i=1}^m dL
 \end{aligned}$$

$$\begin{aligned}
 w &:= w - \frac{\partial J}{\partial w} dw \\
 b &:= b - \frac{\partial J}{\partial b} db
 \end{aligned}$$

1.2.5 向量化

向量化可以消除代码中的for循环，提高速度

$$z = w^T x + b$$

$$w = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix} \quad x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

命令: $z = \text{np.dot}(w, x) + b$

无向量化, 即使用for循环

```

z = 0
for i in range(n):
    z += w[i] * x[i]
z += b

```

$$v = \begin{bmatrix} v_1 \\ \vdots \\ v_n \end{bmatrix} \rightarrow u = \begin{bmatrix} e^{v_1} \\ e^{v_2} \\ \vdots \\ e^{v_n} \end{bmatrix}$$

```

import numpy as np
u = np.exp(v)

```

`np.dot(a, b)` # a矩阵 * b矩阵

`np.log()` # 计算log

`np.abs()` # 计算绝对值

`np.exp()` # 计算指数

`np.maximum()` # 计算最大值

$J = 0, \boxed{dw1 = 0, dw2 = 0}, db = 0$
 $dw = np.zeros((n-x, 1))$
 $\rightarrow$ for i = 1 to n:
 $z^{(i)} = w^T x^{(i)} + b$
 $a^{(i)} = \sigma(z^{(i)})$
 $J += -[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$
 $dz^{(i)} = a^{(i)}(1 - a^{(i)})$
 $\left. \begin{array}{l} dw_1 += x_1^{(i)} dz^{(i)} \\ dw_2 += x_2^{(i)} dz^{(i)} \\ db += dz^{(i)} \end{array} \right\} n_x = 2$
 $dw += x^{(i)} dz^{(i)}$
 $J = J/m, \boxed{dw_1 = dw_1/m, dw_2 = dw_2/m}, db = db/m$
 $\Delta_{1,1} /= m$

Vectorizing Logistic Regression

$\rightarrow \underline{z^{(1)}} = \boxed{w^T x^{(1)} + b}$
 $\underline{z^{(2)}} = \boxed{w^T x^{(2)} + b}$
 $\underline{z^{(3)}} = w^T x^{(3)} + b$
 $\rightarrow \underline{a^{(1)}} = \sigma(z^{(1)})$
 $\underline{a^{(2)}} = \sigma(z^{(2)})$
 $\underline{a^{(3)}} = \sigma(z^{(3)})$

$\bar{X} = \begin{bmatrix} x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ | & | & & | \end{bmatrix}$
 $\begin{matrix} (n_x, m) \\ \mathbb{R}^{n_x \times m} \end{matrix}$
 $\bar{w} = \begin{bmatrix} w^T & 1 \end{bmatrix}$
 $\begin{bmatrix} x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ | & | & & | \end{bmatrix}$

$\underline{Z} = \begin{bmatrix} z^{(1)} & z^{(2)} & \dots & z^{(m)} \end{bmatrix} = \underbrace{\bar{w}^T \bar{X}}_{1 \times m} + \underbrace{\begin{bmatrix} b & b & \dots & b \end{bmatrix}}_{1 \times m} = \begin{bmatrix} \boxed{w^T x^{(1)} + b} & \boxed{w^T x^{(2)} + b} & \dots & \boxed{w^T x^{(m)} + b} \end{bmatrix}$
 $\begin{matrix} \uparrow \\ \bar{Z} \end{matrix}$
 $\begin{matrix} \uparrow \\ \bar{X} \end{matrix}$
 $\bar{Z} = np.dot(\bar{w}, \bar{X}) + \frac{b}{\bar{Z}} \begin{matrix} (1,1) \\ \mathbb{R} \end{matrix}$

"Broadcasting"